**WOXSEN UNIVERSITY**
**SCHOOL OF TECHNOLOGY**

## PROJECT-BASED LEARNING (PBL) REPORT

### DATABASE MANAGEMENT SYSTEMS

# DESIGN AND IMPLEMENTATION OF A DATABASE MANAGEMENT SYSTEM FOR TOURISM PACKAGE AND TRAVEL BOOKING MANAGEMENT SYSTEM

Submitted by: Jashwanth Sappidi

Roll Number: 25WU0102108

Program: B.Tech CSE (AIML)

Section: Rhinos

Date of Submission: 15/10/2026

Academic Year: 2026–2027

Website: https://phillips-pure.vercel.app

# ACKNOWLEDGEMENT

I express my sincere gratitude to Woxsen University, School of Technology, for providing me with the opportunity to undertake this Project-Based Learning activity as part of the Database Management Systems course. I thank my course faculty for guidance and suggestions throughout the project. This work helped me apply ER modelling, relational schema design, normalization, SQL constraints, database transactions, and application development to a practical tourism booking scenario. I also acknowledge the public-facing GlobeTrek website used as the reference interface for package search and tourism information.

# TABLE OF CONTENTS

Abstract

List of Tables

List of Figures

List of Acronyms

1. Problem Statement, Solution Strategy and ER Diagram

1.1 Problem Statement

1.2 Objectives

1.3 Scope and Assumptions

1.4 Proposed Solution Strategy

1.5 Functional Requirements

1.6 Entity-Relationship Diagram (ERD)

1.7 Entity and Relationship Summary

2. Logical Database Design

2.1 Relational Schema

2.2 Keys and Integrity Constraints

2.3 Functional Dependencies

2.4 Normalization up to Third Normal Form (3NF)

2.5 Data Dictionary

2.6 Implementation of Logical Design: DDL and Sample Data

3. Application Development Using Logical Database Design

3.1 Application Overview

3.2 Development Environment and Database Connectivity

3.3 Application Modules and Database Mapping

3.4 CRUD Operations

3.5 Business Rules, Validation and Error Handling

3.6 Reports and Database Operations

3.7 Application Screens

3.8 Testing and Results

4. Conclusion

4.1 Limitations and Future Enhancements

5. References

6. Appendices

# ABSTRACT

Travel agencies manage destinations, tour packages, itineraries, departures, hotels, transport, guides, customers, travellers, bookings, payments, cancellations and refunds. When these records are maintained in separate spreadsheets, duplicate information, overbooking, guide scheduling conflicts and inconsistent financial records can occur. The objective of this project is to design a relational database and application workflow that centralizes tourism package and travel booking information while maintaining data integrity. The proposed design uses an Entity-Relationship model, a relational schema and normalization up to Third Normal Form (3NF). Primary keys, foreign keys, UNIQUE, NOT NULL, CHECK and DEFAULT constraints are used where suitable. Cross-table business rules such as departure capacity, itinerary order, guide availability, confirmed-departure booking and refund limits require application validation and transaction-aware database operations. The proposed Java or Python front end supports package and itinerary management, departure publishing, guide assignment, customer and traveller registration, booking, payment, cancellation, refund and operational reporting. Sample SQL demonstrates joins, subqueries, aggregation and views. The public GlobeTrek website is referenced for the customer-facing tourism search and package presentation. The project provides a structured foundation for safer booking operations, consistent records and useful management reports.



Keywords: Tourism Booking, Relational Database, ER Diagram, Normalization, SQL, Transaction Management

# CHAPTER 1
# PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

## 1.1 Problem Statement

Tourism agencies coordinate information about destinations, packages, itinerary days, departure dates, accommodation, transport, guides and customers. A single booking can include multiple travellers and may involve one or more payments. If these records are stored in separate spreadsheets or manually updated files, staff can accidentally exceed departure capacity, assign one guide to overlapping trips, repeat customer details, or calculate cancellation refunds incorrectly. Finding destination demand, package availability and revenue also becomes time-consuming.

The proposed Tourism Package and Travel Booking Management System organizes these related records in a relational database. The database design provides keys and constraints for data integrity, while application-level checks and transactions enforce rules that involve several records. The linked public website (https://phillips-pure.vercel.app) presents tourism packages and search features and is used as a reference for the customer-facing interface. Its visible public page does not, by itself, verify the existence of a connected administrative database or live payment processing.

## 1.2 Objectives

- Design an ER model and relational schema covering the major tourism entities and their relationships.
- Normalize the relations up to 3NF and document functional dependencies and a data dictionary.
- Implement primary and foreign keys and suitable UNIQUE, NOT NULL, CHECK and DEFAULT constraints.
- Support package search, departure capacity checking, booking, payment, cancellation, refund and management reports.
- Prepare positive and negative tests for data validation, capacity control, guide scheduling and financial integrity.

## 1.3 Scope and Assumptions

| Item | Description |
|---|---|
| In scope | Destinations, packages, itinerary days, departures, hotels, transport, guides, customers, travellers, bookings, payments, cancellations, refunds and reports. |
| In scope | Relational schema, 3NF analysis, SQL DDL/DML, joins, subqueries, aggregates, views, indexes and transaction-aware validation. |
| Out of scope | Live payment-gateway settlement, airline/hotel supplier integrations, identity verification, tax filing and production-scale deployment. |
| Assumptions | Each package belongs to one primary destination; a package can have multiple itinerary days and departures; each departure has a capacity and may have one assigned primary guide. |
| Assumptions | A booking belongs to one customer and one departure and can contain multiple travellers and payment records. Refund amounts are recorded against a cancellation and a payment. |
| Evidence note | The website is public-facing. Interface figures in this report are representative reconstructions from visible page content; they are not browser-captured screenshots or proof of database-backed admin operations. |

## 1.4 Proposed Solution Strategy

| Solution Element | Project-Specific Description |
|---|---|
| Primary users / roles | Administrator or travel-agency staff; booking operator; guide coordinator; customer; report viewer. |
| Major modules | Destination and package setup; itinerary; departures; hotel and transport mapping; guide assignment; customer/traveller; booking; payment; cancellation/refund; reports. |
| Workflow | Create destination and package → add ordered itinerary days → create departure → validate dates/capacity/guide → publish confirmed departure → register customer and travellers → check seats → create booking → record payment → manage cancellation/refund → generate reports. |
| Important business rules | Traveller count must not exceed capacity; itinerary day numbers are positive and unique per package; guide assignments must not overlap; only confirmed departures with available seats can be booked; refunds cannot exceed eligible paid amount. |
| Expected reports | Package availability; departure capacity; booking history; guide schedule; cancellation/refund summary; destination demand; payment and revenue summary. |

## 1.5 Functional Requirements

| Req. ID | Functional Requirement | Priority |
|---|---|---|
| FR-01 | Create, read, update and search destinations, packages and itinerary days. | High |
| FR-02 | Create departures with date, capacity and publication status; validate dates and capacity. | High |
| FR-03 | Assign guides and reject overlapping assignments. | High |
| FR-04 | Register customers and travellers and create bookings only for confirmed departures with available seats. | High |
| FR-05 | Record payments and calculate paid totals for each booking. | High |
| FR-06 | Process cancellations and ensure total refunds do not exceed eligible payment. | High |
| FR-07 | Generate availability, booking, guide schedule, destination demand and revenue reports. | Medium |
| FR-08 | Validate input, use parameterized queries and handle database errors safely. | High |

## 1.6 Entity-Relationship Diagram (ERD)

Figure 1 summarizes the conceptual entities and selected relationships. Associative relations are used for many-to-many relationships such as packages and hotels, and departures and transport services. The final implementation should keep all key names and cardinalities consistent with the approved schema.

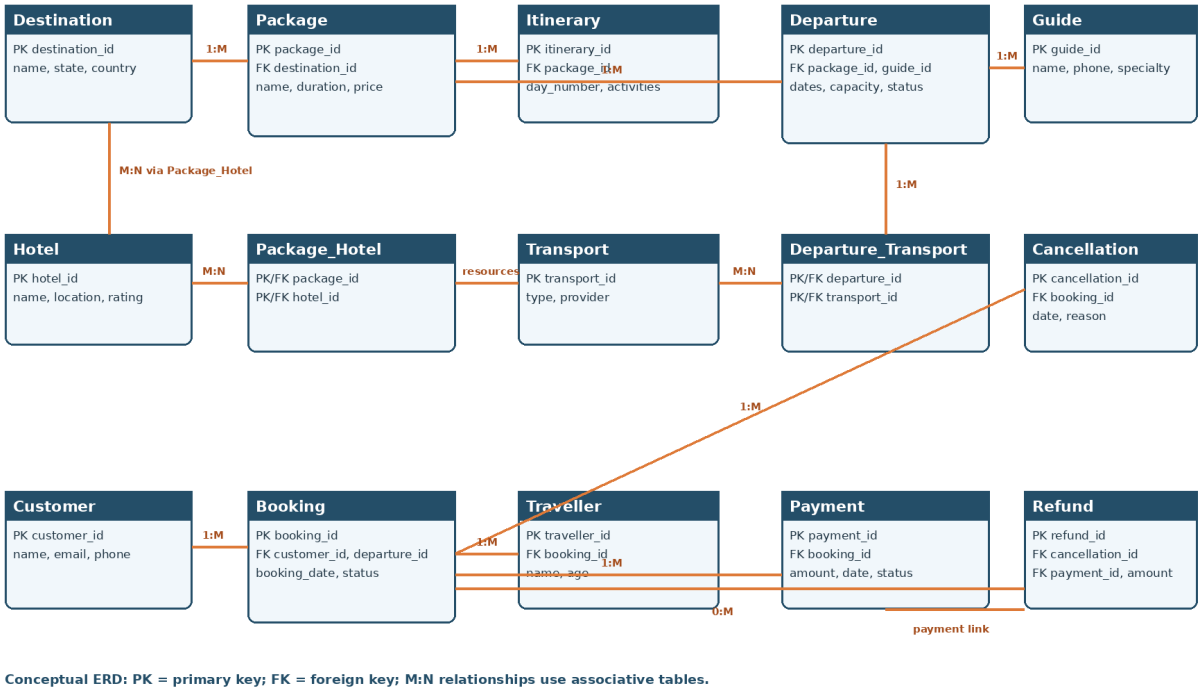



*Figure 1. Entity-Relationship Diagram of the Proposed System*

## 1.7 Entity and Relationship Summary

| Entity / Relationship | Major Attributes / Cardinality | Purpose / Rule |
|---|---|---|
| Destination | destination_id (PK), name, state, country, description; Destination 1:M Package | Stores places offered by the agency. |
| Package | package_id (PK), destination_id (FK), package_name, duration_days, price; Package 1:M Itinerary_Day and Departure | Stores the package catalogue. |
| Itinerary_Day | itinerary_id (PK), package_id (FK), day_number, activities; unique(package_id, day_number) | Stores the ordered daily plan. |
| Departure | departure_id (PK), package_id (FK), guide_id (FK), start_date, end_date, capacity, status | Represents a scheduled batch with limited seats. |
| Guide | guide_id (PK), guide_name, phone, specialization; Guide 1:M Departure over non-overlapping dates | Stores guide details and assignment. |
| Hotel / Transport | hotel_id or transport_id (PK); associative tables link services to packages/departures | Avoids repeating service data in package rows. |
| Customer / Booking / Traveller | Customer 1:M Booking; Booking 1:M Traveller | Separates customer contact details from traveller manifests. |
| Payment / Cancellation / Refund | Booking 1:M Payment; Booking 0:M Cancellation; Cancellation 0:M Refund | Maintains transaction history and financial traceability. |

# CHAPTER 2
# LOGICAL DATABASE DESIGN

## 2.1 Relational Schema

The ER model is converted into relations using primary keys to identify records and foreign keys to maintain relationships. The following schema is the proposed logical design for the project.

| Table Name | Key Attributes | Purpose |
|---|---|---|
| Destination | destination_id PK; name, state, country, description | Destination catalogue |
| Package | package_id PK; destination_id FK; package_name, duration_days, price | Tour package details |
| Itinerary_Day | itinerary_id PK; package_id FK; day_number, activities | Daily itinerary |
| Guide | guide_id PK; guide_name, phone UNIQUE, specialization | Guide directory |
| Departure | departure_id PK; package_id FK; guide_id FK; start_date, end_date, capacity, status | Scheduled batch |
| Hotel | hotel_id PK; hotel_name, city, rating | Hotel directory |
| Package_Hotel | package_id PK/FK; hotel_id PK/FK | Package–hotel mapping |
| Transport | transport_id PK; mode, provider_name, contact_phone | Transport directory |
| Departure_Transport | departure_id PK/FK; transport_id PK/FK | Departure transport mapping |
| Customer | customer_id PK; name, email UNIQUE, phone UNIQUE | Customer contact details |
| Booking | booking_id PK; customer_id FK; departure_id FK; booking_date, status | Booking header |
| Traveller | traveller_id PK; booking_id FK; traveller_name, age | Traveller manifest |
| Payment | payment_id PK; booking_id FK; amount, payment_date, status, method | Payment records |
| Cancellation | cancellation_id PK; booking_id FK; cancellation_date, reason, status | Cancellation history |
| Refund | refund_id PK; cancellation_id FK; payment_id FK; amount, refund_date, status | Refund records |

## 2.2 Keys and Integrity Constraints

| Constraint | Example | Rule Enforced |
|---|---|---|
| PRIMARY KEY | Package(package_id) | Each row has a unique, non-null identifier. |
| FOREIGN KEY | Package.destination_id → Destination.destination_id | A package must refer to an existing destination. |
| NOT NULL | Package.package_name NOT NULL | Required fields cannot be omitted. |
| UNIQUE | Customer.email UNIQUE | Prevents duplicate email addresses. |
| CHECK | price >= 0; capacity > 0; end_date >= start_date | Restricts values to valid ranges. |
| DEFAULT | Departure.status DEFAULT 'DRAFT' | Provides a safe initial status. |
| Composite UNIQUE | UNIQUE(package_id, day_number) | Prevents duplicate itinerary day numbers in one package. |
| Referential actions | RESTRICT for referenced business records; CASCADE only for safe dependent rows | Prevents accidental removal of records needed for audit/history. |

Some rules involve more than one row or table and cannot be safely enforced by a simple row-level CHECK. Traveller capacity, guide overlap, booking status and cumulative refunds must be checked inside a transaction, with appropriate locking or re-checking before commit. A trigger or stored procedure may supplement application validation.

## 2.3 Functional Dependencies

The following functional dependencies are the main determinants used to separate entities and avoid repeated non-key information:

- destination_id → name, state, country, description
- package_id → destination_id, package_name, duration_days, price
- itinerary_id → package_id, day_number, activities; (package_id, day_number) is also unique
- guide_id → guide_name, phone, specialization
- departure_id → package_id, guide_id, start_date, end_date, capacity, status
- customer_id → name, email, phone
- booking_id → customer_id, departure_id, booking_date, status
- traveller_id → booking_id, traveller_name, age
- payment_id → booking_id, amount, payment_date, status, method
- cancellation_id → booking_id, cancellation_date, reason, status
- refund_id → cancellation_id, payment_id, amount, refund_date, status

## 2.4 Normalization up to Third Normal Form (3NF)

| Relation / Group | 1NF | 2NF | 3NF | Reason / Decomposition |
|---|---|---|---|---|
| Destination, Guide, Hotel, Transport, Customer | Yes | Yes | Yes | Atomic attributes; each table uses a single-column key and stores only attributes of that entity. |
| Package | Yes | Yes | Yes | Destination details are stored separately and referenced by destination_id. |
| Itinerary_Day | Yes | Yes | Yes | One row per itinerary day; unique(package_id, day_number) preserves sequence. |
| Departure | Yes | Yes | Yes | Package and guide are referenced by keys; package/guide descriptive attributes are not copied. |
| Package_Hotel, Departure_Transport | Yes | Yes | Yes | Composite key consists of both relationship keys; no non-key attributes depend on only part of the key. |
| Booking, Traveller, Payment | Yes | Yes | Yes | Customer, traveller and payment details are separated to avoid repeating groups and partial/transitive dependencies. |
| Cancellation, Refund | Yes | Yes | Yes | Cancellation and refund records have independent keys and foreign-key links to related transactions. |

1NF is satisfied because each field contains an atomic value and repeating groups are represented by separate rows. 2NF is satisfied because non-key attributes depend on the complete candidate key; this is especially important for the associative tables with composite keys. 3NF is satisfied when non-key attributes depend on the key, the whole key and nothing but the key. For example, destination names are stored in Destination rather than repeated in Package, and customer details are stored in Customer rather than repeated in Booking.

## 2.5 Data Dictionary

| Table | Attribute | Data Type | Key / Constraint | Description |
|---|---|---|---|---|
| Destination | destination_id | INT | PK | Unique destination identifier |
| Destination | name | VARCHAR(100) | NOT NULL | Destination name |
| Destination | state | VARCHAR(80) | NULL allowed | State or region |
| Destination | country | VARCHAR(80) | NOT NULL | Country |
| Package | package_id | INT | PK | Unique package identifier |
| Package | destination_id | INT | FK, NOT NULL | Destination offered |
| Package | package_name | VARCHAR(120) | NOT NULL | Package title |
| Package | duration_days | INT | CHECK > 0 | Duration in days |
| Package | price | DECIMAL(10,2) | CHECK >= 0 | Base price per traveller |
| Itinerary_Day | itinerary_id | INT | PK | Unique itinerary row |
| Itinerary_Day | package_id | INT | FK | Related package |
| Itinerary_Day | day_number | INT | CHECK > 0; UNIQUE with package_id | Day order |
| Departure | departure_id | INT | PK | Unique departure |
| Departure | package_id | INT | FK | Package scheduled |
| Departure | guide_id | INT | FK, nullable | Assigned guide |
| Departure | capacity | INT | CHECK > 0 | Maximum traveller count |
| Departure | status | VARCHAR(20) | DEFAULT 'DRAFT' | Publication/confirmation status |
| Customer | customer_id | INT | PK | Unique customer |
| Customer | email | VARCHAR(150) | UNIQUE | Customer email |
| Booking | booking_id | INT | PK | Unique booking |
| Booking | departure_id | INT | FK | Selected departure |
| Booking | status | VARCHAR(20) | DEFAULT 'PENDING' | Booking status |
| Traveller | traveller_id | INT | PK | Unique traveller record |
| Payment | payment_id | INT | PK | Unique payment |
| Payment | amount | DECIMAL(10,2) | CHECK >= 0 | Payment amount |
| Cancellation | cancellation_id | INT | PK | Unique cancellation |
| Refund | refund_id | INT | PK | Unique refund |
| Refund | amount | DECIMAL(10,2) | CHECK >= 0 | Refund amount |

## 2.6 Implementation of Logical Design: DDL and Sample Data

| Item | Details |
|---|---|
| DBMS | MySQL (proposed implementation target; exact server version must be recorded when run) |
| Number of relations | 15 core and associative relations listed in Section 2.1 |
| Sample data | Illustrative destinations, packages, guides, departures, customers, bookings, payments and refunds |
| Implementation status | DDL and sample SQL are provided in Appendix A. Execution results should be added after running the script in the selected DBMS. |

Representative DDL:

```
CREATE TABLE Destination (
  destination_id INT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  state VARCHAR(80),
  country VARCHAR(80) NOT NULL,
  description VARCHAR(255)
);

CREATE TABLE Package (
  package_id INT PRIMARY KEY,
  destination_id INT NOT NULL,
  package_name VARCHAR(120) NOT NULL,
  duration_days INT NOT NULL CHECK (duration_days > 0),
  price DECIMAL(10,2) NOT NULL CHECK (price >= 0),
  FOREIGN KEY (destination_id)
    REFERENCES Destination(destination_id)
);
```

The complete DDL, illustrative inserts, queries and view definition are included in Appendix A.

Sample data is intended for academic testing and is not claimed to be live production data.

# CHAPTER 3
# APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN

## 3.1 Application Overview

The application is designed around the complete tourism booking lifecycle. Agency staff can maintain destinations and packages, define itinerary days, schedule departures, assign guides, manage customers and travellers, record payments, process cancellations and refunds, and generate reports. The public GlobeTrek website provides customer-facing package search and display content. The review presentations specify a Java or Python front end connected to MySQL; therefore, the code and environment details below describe the proposed integration approach. The public website page alone does not prove that the administrative operations or SQL database are connected to it.

## 3.2 Development Environment and Database Connectivity

| Item | Details |
|---|---|
| Programming language | Python or Java (final implementation choice should be recorded by the team) |
| DBMS | MySQL, as specified in the review presentations |
| IDE / framework / GUI | To be confirmed from the actual implementation; public website is hosted at Vercel |
| Connector | mysql-connector-python for Python, or JDBC driver for Java |
| Operating environment | Development environment to be recorded during implementation |
| Website | https://phillips-pure.vercel.app |
| Repository link | Not supplied in the project material; insert repository URL if available |

Illustrative Python connectivity code (credentials must be configured securely and never committed to source control):

```
import mysql.connector

conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="YOUR_PASSWORD",
    database="tourism_db"
)
cursor = conn.cursor()
print("Database connection established")
```

Database operations should use parameterized queries rather than string concatenation. Booking, payment and refund operations should use transactions so that related updates either commit together or roll back when a validation fails.

## 3.3 Application Modules and Database Mapping

| Module / Screen | Purpose | Tables Used | Main Operations |
|---|---|---|---|
| Destination & Package | Maintain destination catalogue and packages | Destination, Package | Create, read, update, search |
| Itinerary | Store ordered daily activities | Itinerary_Day, Package | Create, update, list by day_number |
| Departure & Guide | Publish departure batches and assign guides | Departure, Guide | Create, validate, update status, schedule check |
| Hotels & Transport | Map accommodation and transport resources | Hotel, Package_Hotel, Transport, Departure_Transport | Create mappings, search resources |
| Customer & Booking | Register customers and bookings | Customer, Booking, Traveller, Departure | Create booking, check seats, view history |
| Payment | Record payment attempts and status | Payment, Booking | Insert payment, summarize paid amount |
| Cancellation & Refund | Record cancellation and refund outcomes | Cancellation, Refund, Payment | Validate eligibility and amount, update status |
| Reports | Provide management summaries | Multiple tables / views | Join, aggregate, filter, export |

## 3.4 CRUD Operations

| Operation | Application Feature / Screen | Tables Affected | Evidence / Expected Behaviour |
|---|---|---|---|
| Create | Add destination/package/ customer/booking | Destination, Package, Customer, Booking, Traveller | Valid form saves a new row and returns its identifier. |
| Read / Search | Browse packages, departures and booking history | Package, Destination, Departure, Booking | Filters return matching records in stable order. |
| Update | Change package details or departure status | Package, Departure, Guide | Validation runs before update; status changes are auditable. |
| Delete / controlled removal | Remove unused draft records only | Relevant master/associative tables | Referenced booking and payment history should not be hard-deleted; use status changes where appropriate. |

## 3.5 Business Rules, Validation and Error Handling

| Rule / Validation | How Implemented | Evidence / Message |
|---|---|---|
| Traveller capacity | Within a transaction, re-check confirmed traveller count and available seats before booking; lock or safely re-check the departure row. | Reject if requested travellers exceed available seats. |
| Itinerary sequence | CHECK day_number > 0 and UNIQUE(package_id, day_number); order output by day_number. | Reject duplicate or non-positive day number. |
| Guide availability | Check assigned guide's existing date intervals before publishing a departure; transactionally re-check concurrent changes. | Reject overlapping assignment and keep departure in draft. |
| Confirmed departure | Booking creation checks departure.status = 'CONFIRMED'. | Reject booking for draft/cancelled departure. |
| Refund limit | Validate refund amount is non-negative and cumulative refund does not exceed eligible payment amount. | Reject over-refund and duplicate refund attempts. |
| Input safety | Required-field validation, parameterized queries and exception handling. | Show a clear validation message without exposing credentials or SQL internals. |
| Transaction handling | Commit only after all related checks pass; rollback on exceptions. | No partial booking/payment/refund changes. |

## 3.6 Reports and Database Operations

| Report / Operation | SQL / Database Concept Used | Application Output |
|---|---|---|
| Package availability | JOIN Package, Destination and Departure; calculate capacity minus confirmed traveller count | Packages with dates, price and remaining seats |
| Customer booking details | JOIN Booking, Customer, Departure and Package | Customer, package, departure date and booking status |
| Destination demand | COUNT bookings grouped by destination; GROUP BY and HAVING | Most requested destinations |
| Revenue summary | SUM verified/paid Payment amounts grouped by package or period | Payment and revenue summary |
| Guide schedule | JOIN Guide and Departure ordered by start_date | Guide assignments and date conflicts |
| Cancellation/refund report | LEFT JOIN Cancellation and Refund; compare refunds with paid amounts | Cancellation status and refund status |
| Confirmed booking view | CREATE VIEW filtered by confirmed booking status | Reusable list of confirmed bookings |
| Nested query | Subquery to find packages priced above the average package price | Package comparison output |

## 3.7 Application Screens

The following figures are representative visual reconstructions based on content available on the linked public website. They are included to document the customer-facing interface concepts. They are not direct browser screenshots and do not demonstrate the back-office CRUD screens,

live database connectivity, or successful payment/refund execution. Replace them with direct screenshots from the running application before final submission if required by the faculty.

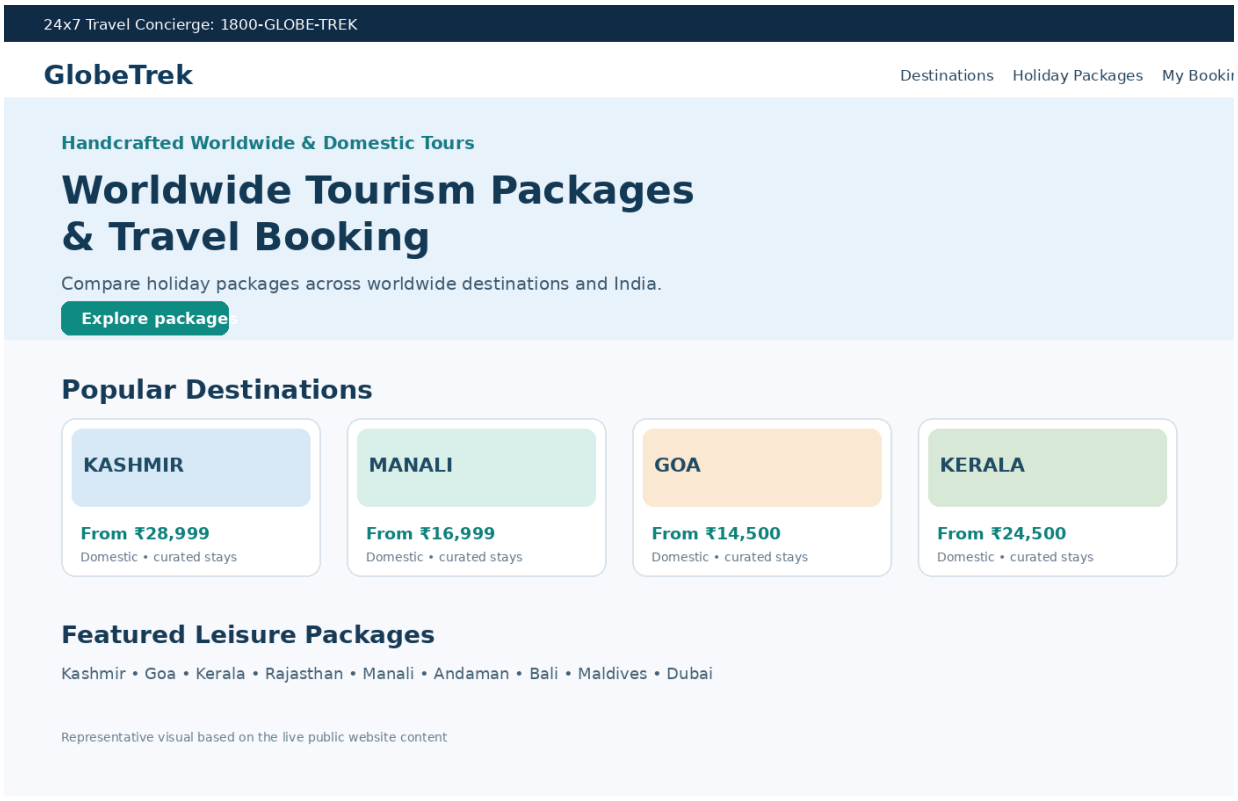



*Figure 2. Website Home Page and Destination Search (representative reconstruction)*

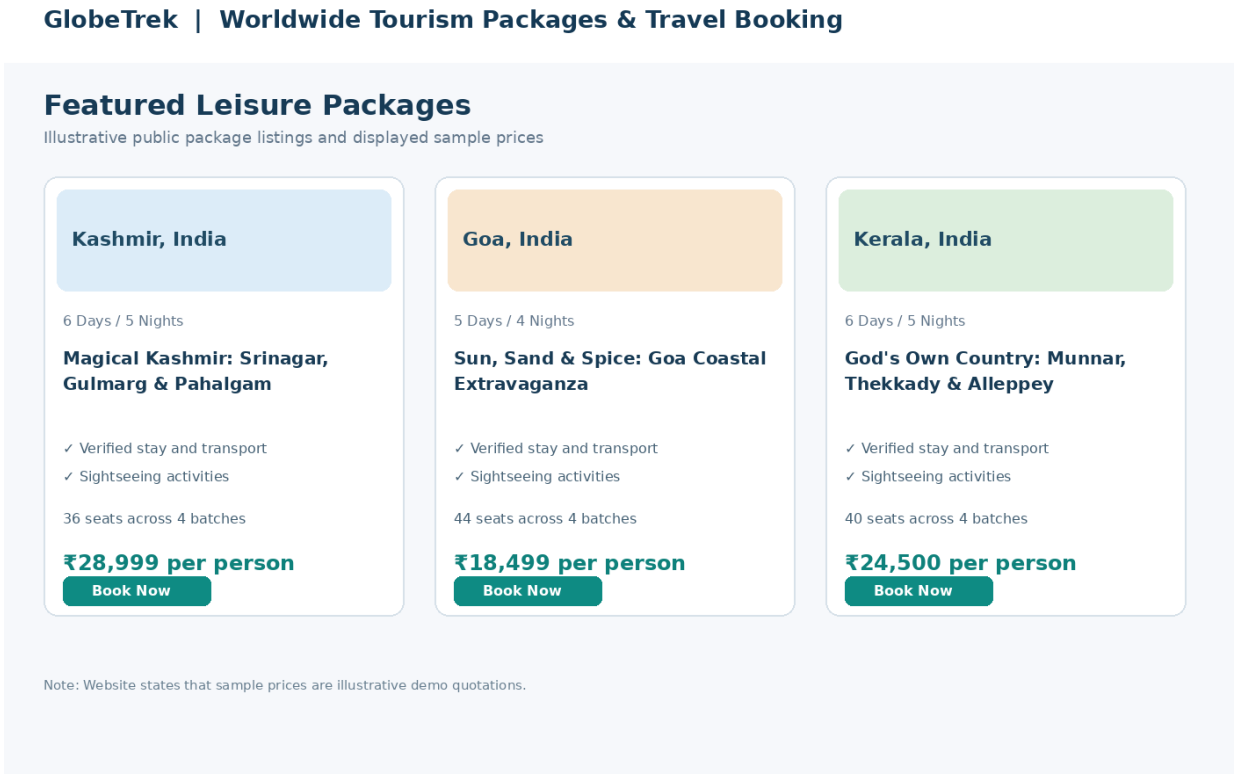


*Figure 3. Featured Tourism Package Listings (representative reconstruction)*

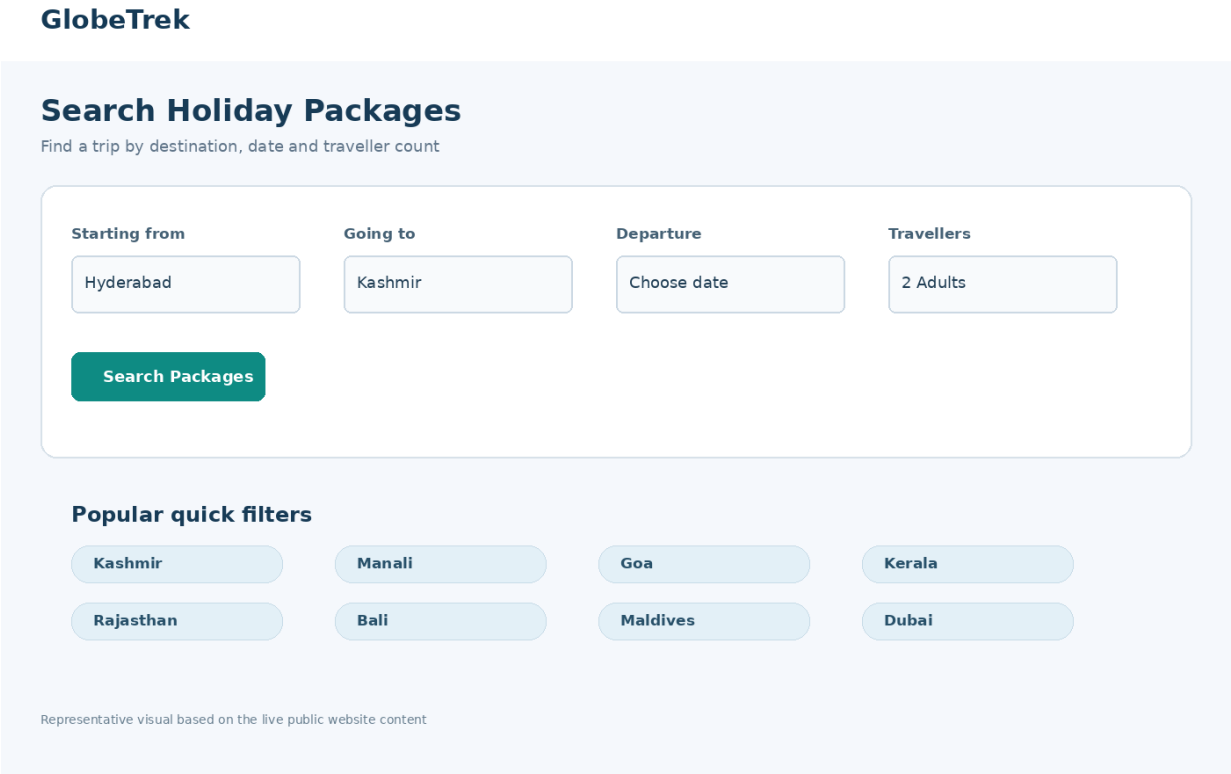



*Figure 4. Package Search Interface (representative reconstruction)*

## Booking Lifecycle and Transaction Workflow

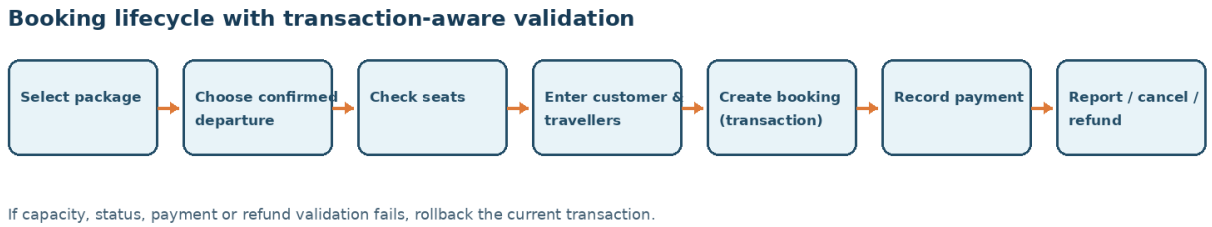



*Figure 5. Booking Lifecycle and Transaction Workflow*

## 3.8 Testing and Results

| Test ID | Feature | Input / Action | Expected Result | Actual Result | Status |
|---|---|---|---|---|---|
| TC-01 | Package creation | Insert package with valid destination and non-negative price | Package is saved | Not run in this report | Pending |
| TC-02 | Primary key | Insert duplicate package_id | Database rejects duplicate key | Not run in this report | Pending |
| TC-03 | Foreign key | Insert package with unknown destination_id | Database rejects invalid reference | Not run in this report | Pending |
| TC-04 | Itinerary | Insert duplicate day_number for same package | Unique constraint rejects row | Not run in this report | Pending |
| TC-05 | Capacity | Book more travellers than available seats | Booking is rejected; no partial rows remain | Not run in this report | Pending |
| TC-06 | Guide schedule | Assign same guide to overlapping departure dates | Assignment is rejected | Not run in this report | Pending |
| TC-07 | Booking status | Attempt booking for DRAFT departure | Booking is rejected | Not run in this report | Pending |
| TC-08 | Refund limit | Enter refund greater than eligible paid amount | Refund is rejected | Not run in this report | Pending |
| TC-09 | Search/report | Run destination-demand aggregation | Counts match source booking records | Not run in this report | Pending |

Testing note: the supplied review materials define the intended rules and test plan but do not provide observed execution logs for these cases. Therefore, actual results are marked “Not run in this report” and status “Pending” rather than claiming unverified success. After running the database and application, update the Actual Result and Status columns with observed evidence.

# CHAPTER 4
# CONCLUSION

The Tourism Package and Travel Booking Management System provides a structured database design for the main information used by a travel agency. The proposed schema separates destinations, packages, itinerary days, departures, guides, customers, bookings, travellers, payments, cancellations and refunds, with supporting hotel and transport mappings. Functional dependencies and normalization up to 3NF reduce duplication and help prevent insertion, update and deletion anomalies. Primary and foreign keys and domain constraints provide a foundation for reliable records.

The application workflow connects package and itinerary setup to departure publishing, booking, payment, cancellation/refund and reporting. Rules that span multiple rows—particularly capacity, guide overlap and refund limits—require transaction-aware validation in addition to declarative constraints. The public GlobeTrek website demonstrates customer-facing package search and display, while the administrative database integration and test results should be verified in the actual implementation. The project reinforces practical use of ER modelling, SQL, normalization, constraints, transactions and report generation.

## 4.1 Limitations and Future Enhancements

- The report uses illustrative sample data and representative reconstructed website views; direct screenshots and test execution evidence should be added from the running system.
- Live payment gateway integration, supplier APIs, authentication roles, audit logging and deployment security are outside the current documented scope.
- Future work can add automated departure reminders, real-time seat updates, secure customer login, payment reconciliation, cancellation policy configuration and exportable dashboards.
- Indexes should be selected after observing query patterns; candidate indexes include Departure(start_date, status), Booking(customer_id, status), Booking(departure_id), Payment(booking_id, status) and Package(destination_id, price).

# 5. REFERENCES

[1] Oracle, “MySQL 8.4 Reference Manual,” MySQL Documentation. Accessed: Oct. 14, 2026. [Online]. Available: https://dev.mysql.com/doc/

[2] Oracle, “MySQL Connector/Python Developer Guide,” MySQL Documentation. Accessed: Oct. 14, 2026. [Online]. Available: https://dev.mysql.com/doc/connector-python/en/

[3] PostgreSQL Global Development Group, “Constraints,” PostgreSQL Documentation. Accessed: Oct. 14, 2026. [Online]. Available: https://www.postgresql.org/docs/current/ddl-constraints.html

[4] Woxsen University, School of Technology, “DBMS PBL Report Preparation Guidelines,” course template supplied for the project, Academic Year 2026–2027.

[5] GlobeTrek Holidays, “Worldwide Tourism Packages & Travel Booking,” public website. Accessed: Oct. 14, 2026. [Online]. Available: https://phillips-pure.vercel.app/

[6] J. Sappidi, “Tourism Package and Travel Booking Management System,” DBMS PBL Review 2 presentation, Academic Year 2026–2027, supplied project material.

[7] J. Sappidi, “Tourism Package and Travel Booking Management System,” DBMS PBL Review 3 presentation, Academic Year 2026–2027, supplied project material.

# 6. APPENDICES

## APPENDIX A – COMPLETE DATABASE SCRIPT

The following MySQL script is a representative starting point for the logical schema. Run it in a test database, validate it against the installed MySQL version, and expand it if the final implementation contains additional attributes.

```
CREATE DATABASE IF NOT EXISTS tourism_db;
USE tourism_db;

CREATE TABLE Destination (
  destination_id INT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  state VARCHAR(80),
  country VARCHAR(80) NOT NULL,
  description VARCHAR(255)
);

CREATE TABLE Guide (
  guide_id INT PRIMARY KEY,
  guide_name VARCHAR(100) NOT NULL,
  phone VARCHAR(20) UNIQUE,
  specialization VARCHAR(100)
);

CREATE TABLE Package (
  package_id INT PRIMARY KEY,
  destination_id INT NOT NULL,
  package_name VARCHAR(120) NOT NULL,
  duration_days INT NOT NULL CHECK (duration_days > 0),
  price DECIMAL(10,2) NOT NULL CHECK (price >= 0),
  FOREIGN KEY (destination_id) REFERENCES Destination(destination_id)
);

CREATE TABLE Itinerary_Day (
  itinerary_id INT PRIMARY KEY,
  package_id INT NOT NULL,
  day_number INT NOT NULL CHECK (day_number > 0),
  activities VARCHAR(255) NOT NULL,
  UNIQUE (package_id, day_number),
  FOREIGN KEY (package_id) REFERENCES Package(package_id)
);

CREATE TABLE Departure (
  departure_id INT PRIMARY KEY,
  package_id INT NOT NULL,
  guide_id INT,
  start_date DATE NOT NULL,
  end_date DATE NOT NULL,
  capacity INT NOT NULL CHECK (capacity > 0),
  status VARCHAR(20) NOT NULL DEFAULT 'DRAFT',
  CHECK (end_date >= start_date),
  FOREIGN KEY (package_id) REFERENCES Package(package_id),
  FOREIGN KEY (guide_id) REFERENCES Guide(guide_id)
);

CREATE TABLE Hotel (
  hotel_id INT PRIMARY KEY,
  hotel_name VARCHAR(120) NOT NULL,
  city VARCHAR(80) NOT NULL,
  rating DECIMAL(2,1) CHECK (rating BETWEEN 1 AND 5)
);
```

```sql
CREATE TABLE Package_Hotel (
  package_id INT NOT NULL,
  hotel_id INT NOT NULL,
  PRIMARY KEY (package_id, hotel_id),
  FOREIGN KEY (package_id) REFERENCES Package(package_id),
  FOREIGN KEY (hotel_id) REFERENCES Hotel(hotel_id)
);

CREATE TABLE Transport (
  transport_id INT PRIMARY KEY,
  mode VARCHAR(40) NOT NULL,
  provider_name VARCHAR(120) NOT NULL,
  contact_phone VARCHAR(20)
);

CREATE TABLE Departure_Transport (
  departure_id INT NOT NULL,
  transport_id INT NOT NULL,
  PRIMARY KEY (departure_id, transport_id),
  FOREIGN KEY (departure_id) REFERENCES Departure(departure_id),
  FOREIGN KEY (transport_id) REFERENCES Transport(transport_id)
);

CREATE TABLE Customer (
  customer_id INT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) UNIQUE,
  phone VARCHAR(20) UNIQUE
);

CREATE TABLE Booking (
  booking_id INT PRIMARY KEY,
  customer_id INT NOT NULL,
  departure_id INT NOT NULL,
  booking_date DATE NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'PENDING',
  FOREIGN KEY (customer_id) REFERENCES Customer(customer_id),
  FOREIGN KEY (departure_id) REFERENCES Departure(departure_id)
);

CREATE TABLE Traveller (
  traveller_id INT PRIMARY KEY,
  booking_id INT NOT NULL,
  traveller_name VARCHAR(100) NOT NULL,
  age INT CHECK (age >= 0),
  FOREIGN KEY (booking_id) REFERENCES Booking(booking_id)
);

CREATE TABLE Payment (
  payment_id INT PRIMARY KEY,
  booking_id INT NOT NULL,
  amount DECIMAL(10,2) NOT NULL CHECK (amount >= 0),
  payment_date DATE NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'PENDING',
  method VARCHAR(40),
  FOREIGN KEY (booking_id) REFERENCES Booking(booking_id)
);

CREATE TABLE Cancellation (
  cancellation_id INT PRIMARY KEY,
  booking_id INT NOT NULL,
  cancellation_date DATE NOT NULL,
  reason VARCHAR(255),
  status VARCHAR(20) NOT NULL DEFAULT 'REQUESTED',
  FOREIGN KEY (booking_id) REFERENCES Booking(booking_id)
);
```

```
CREATE TABLE Refund (
  refund_id INT PRIMARY KEY,
  cancellation_id INT NOT NULL,
  payment_id INT NOT NULL,
  amount DECIMAL(10,2) NOT NULL CHECK (amount >= 0),
  refund_date DATE,
  status VARCHAR(20) NOT NULL DEFAULT 'PENDING',
  FOREIGN KEY (cancellation_id) REFERENCES Cancellation(cancellation_id),
  FOREIGN KEY (payment_id) REFERENCES Payment(payment_id)
);

-- Illustrative sample records
INSERT INTO Destination VALUES
(1,'Goa','Goa','India','Coastal tourism'),
(2,'Kerala','Kerala','India','Backwaters and hill stations'),
(3,'Kashmir','Jammu and Kashmir','India','Mountain and lake tourism');

INSERT INTO Guide VALUES
(1,'Arun Kumar','9000000001','Heritage and city tours'),
(2,'Meera Nair','9000000002','Nature and backwaters');

INSERT INTO Package VALUES
(101,1,'Goa Coastal Extravaganza',5,18499.00),
(102,2,'Kerala Backwater Escape',6,24500.00),
(103,3,'Magical Kashmir',6,28999.00);

INSERT INTO Itinerary_Day VALUES
(1,101,1,'Arrival and beach visit'),
(2,101,2,'Old Goa heritage tour'),
(3,102,1,'Arrival in Kochi'),
(4,102,2,'Munnar sightseeing');

INSERT INTO Departure VALUES
(1001,101,1,'2026-11-10','2026-11-14',30,'CONFIRMED'),
(1002,102,2,'2026-12-01','2026-12-06',25,'CONFIRMED'),
(1003,103,1,'2026-12-10','2026-12-15',20,'DRAFT');

INSERT INTO Customer VALUES
(501,'Aarav Sharma','aarav@example.com','9880000001'),
(502,'Diya Reddy','diya@example.com','9880000002');

INSERT INTO Booking VALUES
(9001,501,1001,'2026-10-01','CONFIRMED'),
(9002,502,1002,'2026-10-03','PENDING');

INSERT INTO Traveller VALUES
(1,9001,'Aarav Sharma',21),
(2,9001,'Anika Sharma',19),
(3,9002,'Diya Reddy',20);

INSERT INTO Payment VALUES
(7001,9001,36998.00,'2026-10-01','PAID','UPI'),
(7002,9002,10000.00,'2026-10-03','PENDING','UPI');

-- Example: customer booking details
SELECT c.name, p.package_name, d.start_date, b.status
FROM Booking b
JOIN Customer c ON c.customer_id = b.customer_id
JOIN Departure d ON d.departure_id = b.departure_id
JOIN Package p ON p.package_id = d.package_id;

-- Example: package availability (illustrative confirmed bookings)
SELECT d.departure_id, p.package_name, d.capacity,
       COALESCE(x.booked_travellers, 0) AS booked_travellers
-- count confirmed travellers from bookings only
FROM Departure d
```

```
JOIN Package p ON p.package_id = d.package_id
LEFT JOIN (
  SELECT b2.departure_id, COUNT(t2.traveller_id) AS booked_travellers
  FROM Booking b2 JOIN Traveller t2 ON t2.booking_id = b2.booking_id
) x ON x.departure_id = d.departure_id


GROUP BY d.departure_id, p.package_name, d.capacity;

-- Example: destination demand
SELECT dst.name, COUNT(b.booking_id) AS booking_count
FROM Destination dst
JOIN Package p ON p.destination_id = dst.destination_id
JOIN Departure d ON d.package_id = p.package_id
LEFT JOIN Booking b ON b.departure_id = d.departure_id
GROUP BY dst.destination_id, dst.name
ORDER BY booking_count DESC;

-- Example: payment summary
SELECT b.booking_id, c.name, SUM(CASE WHEN pay.status = 'PAID'
                                      THEN pay.amount ELSE 0 END) AS total_paid
FROM Booking b
JOIN Customer c ON c.customer_id = b.customer_id
LEFT JOIN Payment pay ON pay.booking_id = b.booking_id
GROUP BY b.booking_id, c.name;

-- Example: confirmed bookings view
CREATE VIEW ConfirmedBookings AS
SELECT b.booking_id, c.name AS customer_name, d.start_date,
       p.package_name, b.status
FROM Booking b
JOIN Customer c ON c.customer_id = b.customer_id
JOIN Departure d ON d.departure_id = b.departure_id
JOIN Package p ON p.package_id = d.package_id
WHERE b.status = 'CONFIRMED';

-- Suggested indexes for frequent search patterns
CREATE INDEX idx_departure_date_status ON Departure(start_date, status);
CREATE INDEX idx_booking_customer_status ON Booking(customer_id, status);
CREATE INDEX idx_payment_booking_status ON Payment(booking_id, status);
```

## APPENDIX B – APPLICATION SOURCE CODE

Website reference: https://phillips-pure.vercel.app

Repository URL: Not provided in the supplied material. Add the GitHub repository or source-code folder path here before submission.

For the final submission, include the actual application source code and configuration instructions. Do not include database passwords or other secrets.

## APPENDIX C – ADDITIONAL TEST DATA AND OUTPUTS

Attach direct screenshots of the running application after executing the test cases in Section 3.8. Recommended evidence includes: (1) package or itinerary form with saved data, (2) departure search and remaining-seat display, (3) booking confirmation with traveller list, (4) payment or cancellation/refund record, and (5) at least one SQL report such as destination demand or revenue summary.

Current report figures are representative reconstructions based on the publicly accessible website content, not direct browser captures. Replace them with real screenshots if the marking rubric requires screenshots of the live page.